

JAVA LAB WEEK-6

1. Basic Information

Title: Find the Longest Substring Without Repeating Characters

Participated Topics:

- Static fields and methods
 - this reference
 - final keyword
 - String handling
-

2. Objective / Aim

To write a Java program to find the **longest substring without repeating characters** in a given string.

The program also demonstrates:

- Static fields and static methods
- this reference
- final keyword
- String handling
- Classes, objects, constructors, and methods

Examples

Input: abcabcb

Output: 3

Longest substring: abc

Input: pwwkew

Output: 3

Longest substring: wke

Note: pwke is not a substring because its characters are not continuous in the original string. It is a subsequence.

3. System Requirements

Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM
- Minimum 1 GB free storage
- Keyboard and Monitor

Software Requirements

- Operating System: Windows / Linux / macOS
 - Java Development Kit: **JDK 17 or JDK 21**
 - IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
 - Command Prompt or Terminal
-

4. Algorithm / Logic Description

1. Start the program.
2. Create a class named LongestSubstring.
3. Declare a **static field** to count the number of objects created.
4. Declare a **final constant** to represent the size of the character table.
5. Create a constructor using the this reference to initialize the input string.
6. Create an object of the LongestSubstring class.
7. Read the input string from the user.
8. Use a static method named findLongestSubstring() to find the longest substring.
9. Maintain two pointers, left and right, to represent the current substring.
10. Use an integer array to store the latest position of each character.
11. When a repeated character is found, move the left pointer forward.
12. Calculate the length of the current substring.
13. Store the starting position and length of the longest substring.
14. Extract the longest substring using the substring() method.
15. Display the longest substring and its length.
16. Stop the program.

OOP Concepts Used

- **Class:** LongestSubstring
- **Object:** obj
- **Attribute:** text
- **Constructor:** LongestSubstring(String text)
- **Instance method:** displayResult()
- **Static field:** objectCount
- **Static method:** findLongestSubstring()
- **Final keyword:** CHARACTER_COUNT
- **This reference:** this.text

Inheritance and polymorphism are not required for this experiment.

5. UML Class Diagram

```

+-----+
|      LongestSubstring      |
+-----+
| - text : String           |
| - objectCount : int {static}      |
| - CHARACTER_COUNT : int {final, static} = 256 |
+-----+
| + LongestSubstring(text : String)      |
| + findLongestSubstring(s : String) : String |
| {static}                               |
| + displayResult() : void              |
+-----+

```

Object Creation

```

|
v

```

```
+-----+
|  obj  |
| LongestSubstring |
+-----+
```

6. Source Code

```
import java.util.Scanner;

public class LongestSubstring {

    // Instance field to store the input string
    private String text;

    // Static field to count the number of objects created
    private static int objectCount = 0;

    // Final constant representing the ASCII character count
    private static final int CHARACTER_COUNT = 256;

    // Constructor using this reference
    LongestSubstring(String text) {
        this.text = text;
        objectCount++;
    }

    // Static method to find the longest substring without repeating characters
    public static String findLongestSubstring(String s) {
```

```
int[] lastIndex = new int[CHARACTER_COUNT];

// Initialize all positions to -1
for (int i = 0; i < CHARACTER_COUNT; i++) {
    lastIndex[i] = -1;
}

int left = 0;
int maxLength = 0;
int start = 0;

for (int right = 0; right < s.length(); right++) {

    char ch = s.charAt(right);

    // Move left pointer if the character is repeated
    if (lastIndex[ch] >= left) {
        left = lastIndex[ch] + 1;
    }

    lastIndex[ch] = right;

    // Find the maximum substring length
    if (right - left + 1 > maxLength) {
        maxLength = right - left + 1;
        start = left;
    }
}
```

```
        return s.substring(start, start + maxLength);
    }

    // Instance method to display the result
    public void displayResult() {

        String longest = findLongestSubstring(this.text);

        System.out.println("Input String    : " + this.text);
        System.out.println("Longest Substring : " + longest);
        System.out.println("Length      : " + longest.length());
        System.out.println("Objects Created : " + objectCount);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Read the input string
        System.out.print("Enter a string: ");
        String input = sc.nextLine();

        // Create an object using the constructor
        LongestSubstring obj = new LongestSubstring(input);

        // Display the result
        obj.displayResult();
    }
}
```

```
        sc.close();  
    }  
}
```

7. Compilation & Execution Commands

Java File Name

LongestSubstring.java

Compilation Command

javac LongestSubstring.java

Execution Command

java LongestSubstring

8. Output Screen

Sample Output 1

Input:

Enter a string: abcabcbbb

Output:

Input String : abcabcbbb

Longest Substring : abc

Length : 3

Objects Created : 1

Sample Output 2

Input:

Enter a string: pwwkew

Output:

Input String : pwwkew

Longest Substring : wke

Length : 3

Objects Created : 1

Observation

For abcabcbb, the longest substring without repeating characters is abc, whose length is 3.

For pwwkew, the longest substring without repeating characters is wke, whose length is 3.

pwke is not considered because the characters are not continuous in the original string. Therefore, pwke is a **subsequence**, not a substring.

9. Result

The Java program to find the **longest substring without repeating characters** was successfully implemented and executed.

The experiment demonstrated:

- Static fields
- Static methods
- this reference
- final keyword
- String handling
- Classes, objects, constructors, attributes, and methods